



Mysten Securitize

Security Assessment

April 14th, 2026 — Prepared by OtterSec

Michał Bochnak

embe221ed@osec.io

Sangsoo Kang

sangsoo@osec.io

Thiago Tavares

thitav@osec.io

Table of Contents

Executive Summary	3
Overview	3
Key Findings	3
Scope	4
Findings	5
Vulnerabilities	6
OS-MSC-ADV-00 Misordered Holding Limits Arguments	8
OS-MSC-ADV-01 Unclaimed Publisher Blocks Migration	9
OS-MSC-ADV-02 Improper Cleanup Due to Backdated Timestamps	10
OS-MSC-ADV-03 Transfer Context Pollutes PartyInfo	11
OS-MSC-ADV-04 Rule Wrapper Missing Token Binding	12
OS-MSC-ADV-05 Improper SetServiceOwner Access Control	13
OS-MSC-ADV-06 Inconsistency in Flowback Activation Logic	14
OS-MSC-ADV-07 Unvalidated Role Cleanup Ability	15
OS-MSC-ADV-08 Missing Zero Check on TreasuryCap Total Supply	16
General Findings	17
OS-MSC-SUG-00 Unreachable Seize Recipient Branch	18
OS-MSC-SUG-01 Docstring and Implementation Mismatch	19
OS-MSC-SUG-02 Missing Issuance Time Validation	21
OS-MSC-SUG-03 Possibility to Assign Wallet Type to Zero	22
OS-MSC-SUG-04 Incorrect Percentage Limit Calculation	23
OS-MSC-SUG-05 Code Refactoring	24

OS - MSC - SUG - 06 Code Maturity	26
-------------------------------------	----

Appendices

Vulnerability Rating Scale	28
-----------------------------------	-----------

Procedure	29
------------------	-----------

01 — Executive Summary

Overview

Mysten labs engaged OtterSec to assess the `securitize` program. This assessment was conducted between March 10th and March 27th, 2026. For more information on our auditing methodology, refer to [Appendix B](#).

Key Findings

We produced 16 findings throughout this audit engagement.

In particular, we identified a vulnerability in the function responsible for removing a role type that may be abused to remove critical abilities due to a lack of type validation ([OS-MS-C-ADV-07](#)). Additionally, there is no check to ensure that the TreasuryCap's total supply is zero during initialization, which may result in unexpected token states ([OS-MS-C-ADV-08](#)). Also, when backdating is allowed, utilizing the issuance timestamp for cleanup may leave expired issuances uncleared, as it does not reflect the current time, resulting in stale issuance history ([OS-MS-C-ADV-02](#)).

The rule wrappers are also not bound to a specific token type, allowing rules to be reutilized across unrelated tokens ([OS-MS-C-ADV-04](#)). Furthermore, when setting the service owner, the logic assumes only the Master has SetServiceOwner ability, but this is not enforced. Granting this ability to other roles may result in the function aborting and breaking ownership transfer logic ([OS-MS-C-ADV-05](#)).

We also made recommendations for more flexible approval rules, clearly distinguishing missing data from a real default status, and improved access control logic ([OS-MS-C-SUG-05](#)). We further suggested to remove redundant code, ensure alignment between solidity and move implementations, and to add functionality to update investor counts when compliance metadata changes ([OS-MS-C-SUG-06](#)). We also advised to check that the issuance time is never greater than the current block time ([OS-MS-C-SUG-02](#)).

02 — Scope

The source code was delivered to us in Git repositories at <https://github.com/securitize-io/bc-sui-dstoken-sc>. This audit was performed against commit [eb5076d](#).

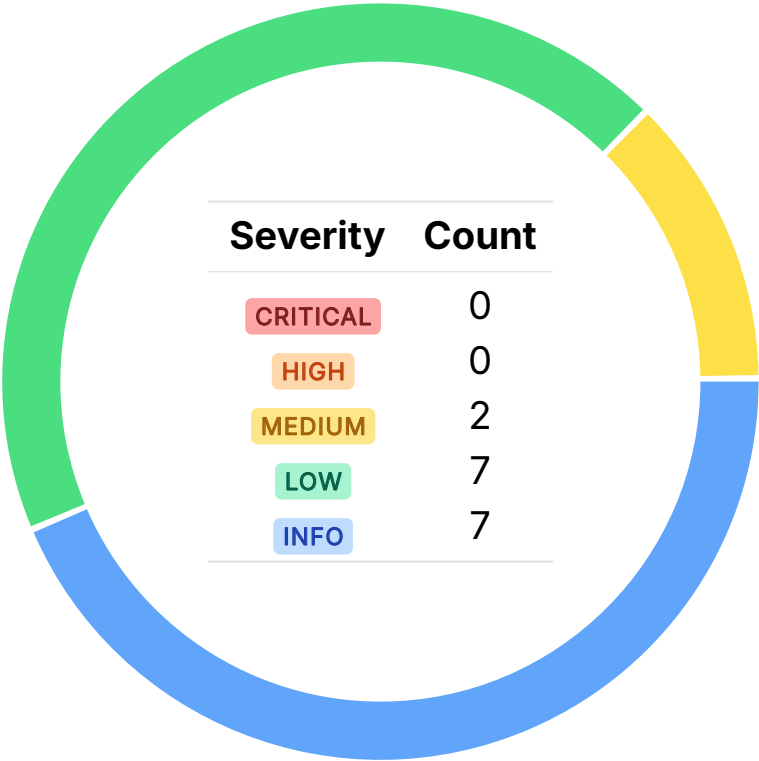
A brief description of the program is as follows:

Name	Description
securitize	A permissioned security-token protocol on Sui that lets authorized parties issue, transfer, burn, and seize tokens while using PAS for custody and approvals. It maintains an investor registry and enforces configurable compliance rules like accreditation, holding/investor limits, lockups, country restrictions, and special-wallet permissions on every token operation.

03 — Findings

Overall, we reported 16 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in [Appendix A](#).

ID	Severity	Status	Description
OS-MSC-ADV-00	MEDIUM	RESOLVED ✓	<code>HoldingLimits</code> transfer validation is called with misordered arguments, so it checks the wrong region and the wrong recipient balance.
OS-MSC-ADV-01	MEDIUM	RESOLVED ✓	<code>migrate</code> relies on an unclaimed <code>Publisher</code> , rendering it impractical to invoke and blocking version updates.
OS-MSC-ADV-02	LOW	RESOLVED ✓	When backdating is allowed, utilizing <code>issuance.timestamp_ms</code> for cleanup may result in expired issuances remaining because it does not reflect the current time.
OS-MSC-ADV-03	LOW	RESOLVED ✓	<code>validate_transfer</code> derives <code>PartyInfo</code> via the transfer amount, so <code>is_exiting</code> field reflect the proposed transfer rather than the current baseline state.
OS-MSC-ADV-04	LOW	RESOLVED ✓	The rule wrappers are not bound to a specific token type, allowing rules to be reutilized across unrelated tokens.
OS-MSC-ADV-05	LOW	RESOLVED ✓	<code>set_service_owner</code> assumes only the <code>Master</code> has <code>SetServiceOwner</code> , but this is not enforced. Granting this ability to other roles may result in the function aborting and breaking ownership transfer logic.

OS-MSC-ADV-06	LOW	RESOLVED ✓	<code>is_active</code> returns <code>false</code> when <code>end == 0</code> , but <code>validate_rule</code> treats it as always active. This creates an inconsistency between reported and actual enforcement state.
OS-MSC-ADV-07	LOW	RESOLVED ✓	<code>remove_role_type</code> does not validate that <code>A</code> is the correct management ability for role <code>R</code> , allowing an authorized caller to remove unrelated critical abilities from <code>Master</code> during role cleanup.
OS-MSC-ADV-08	LOW	RESOLVED ✓	There is no check to ensure that the <code>TreasuryCap</code> 's total supply is zero during initialization, which may result in unexpected token states.

Misordered Holding Limits Arguments

MEDIUM

OS-MSC-ADV-00

Description

`compliance_service::validate_transfer_rule` passes the arguments to `HoldingLimits::validate_holding_limits_for_transfer` in the wrong order, swapping `to.balance` and `from.region`. As a result, sender region-based minimum holding checks may silently fail, because the function receives a token balance where it expects a region code. Also, the receiver's post-transfer balance may be computed utilizing a small region constant such as 1 or 2, instead of the receiver's real balance.

```
>_ move/securitize/sources/compliance_service.move
```

RUST

```
/// Validate a single transfer rule
fun validate_transfer_rule<T>(
    config: &ComplianceConfig<T>,
    registry: &InvestorInfo<T>,
    rule: TypeName,
    transfer: &TransferInfo,
    from: &PartyInfo,
    to: &PartyInfo,
    from_issuances_data: &vector<Issuance>,
) {
    [...]
} else if (rule == type_name::with_defining_ids<HoldingLimits>()) {
    let rule: &HoldingLimits = config.rules_bag.borrow(rule);
    rule.validate_holding_limits_for_transfer(
        transfer.amount,
        from.is_special_wallet,
        from.balance,
        to.balance,
        from.region,
        to.region,
    );
}
[...]
```

Remediation

Ensure the arguments are passed in correct order to `validate_holding_limits_for_transfer`.

Patch

Resolved in [99fe56d](#).

Unclaimed Publisher Blocks Migration

MEDIUM

OS-MS-C-ADV-01

Description

`version::migrate` depends on a `Publisher` reference to authorize version updates via `pub.from_package<Version>()`, but if this object is not claimed by the protocol, no entity may reliably invoke it. This renders the migration path impractical, as the required authority cannot be exercised in production. As a result, the `Version` object may never be updated after an upgrade.

```
>_ move/securitize/sources/version.move
```

RUST

```
public fun migrate(pub: &Publisher, version: &mut Version) {  
    assert!(pub.from_package<Version>(), EInvalidPublisher);  
    version.version = VERSION;  
}
```

Remediation

Utilize a controlled authority such as `SetupRegistry.admin` to ensure that a known role within the system may perform migrations.

Patch

Resolved in [478a9fd](#).

Improper Cleanup Due to Backdated Timestamps

LOW

OS-MSC-ADV-02

Description

`compliance_service::record_issuance` passes `issuance.timestamp_ms` into `cleanup_party_issuances`, but when backdating is enabled, this timestamp may be in the past relative to the actual execution time. As a result, cleanup is performed with a non-current reference time rather than the actual current time. This may result in incorrect retention of expired issuances. Entries that should already be considered unlocked (based on the current time) may not be removed because the cleanup compares lock expiry against an older, backdated timestamp.

```
>_ move/securitize/sources/compliance_service.move
```

RUST

```
public(package) fun record_issuance<T>(  
    config: &ComplianceConfig<T>,  
    registry: &mut InvestorInfo<T>,  
    issuance: &IssuanceInfo,  
    to: &PartyInfo,  
) {  
    if (issuance.amount == 0) return;  
    [...]  
    cleanup_party_issuances(config, registry, to, issuance.timestamp_ms);  
    emit_compliance_issuance_recorded_event<T>(to.addr, issuance.amount);  
}
```

Remediation

Utilize the current timestamp instead of `issuance.timestamp_ms` to ensure that expiry is evaluated against real time.

Patch

Resolved in [b8720fd](#).

Transfer Context Pollutes PartyInfo

LOW

OS-MS-C-ADV-03

Description

`compliance_service::validate_transfer` passes the `amount` into `get_party_info`, resulting in `is_exiting` field to be derived utilizing the proposed transfer delta rather than the current on-chain state. As a result, the `is_exiting` field in `to_info` was set to an incorrect value.

```
>_ move/securitize/sources/compliance_service.move
```

RUST

```
public(package) fun validate_transfer<T>(
  config: &ComplianceConfig<T>,
  registry: &mut InvestorInfo<T>,
  request: &Request<SendFunds<Balance<T>>>,
  current_time_ms: u64,
  version: &Version,
) {
  version.check_is_valid();

  let from_address = request.data().sender();
  let to_address = request.data().recipient();
  let amount = request.data().funds().value();
  [...]
  // Get party info for both sides
  let mut from_info = get_party_info(registry, from_address, amount);
  let to_info = get_party_info(registry, to_address, amount);
  [...]
}
```

Remediation

Pass `0` instead of `amount` to `get_party_info` to keep `PartyInfo` as a current-state snapshot and avoid mixing transition effects into the general compliance context.

Patch

Resolved in [b8720fd](#).

Rule Wrapper Missing Token Binding

LOW

OS-MSC-ADV-04

Description

In `rule_wrapper`, `RuleInitWrapper<T>` and `RuleUpdateWrapper<T>` are only parameterized by the rule type `T`, with no binding to a specific token type. This allows a rule instantiated under one token's authority context to be passed and registered for another token without restriction. As a result, type-level guarantees about rule-token coupling are lost, weakening isolation between assets. This may result in incorrect enforcement of compliance policies across tokens.

```
>_ move/securitize/sources/rule_wrapper.move
```

RUST

```
/// Hot potato wrapper used during *rule creation*.
/// Must be resolved (unwrapped) in the same transaction.
public struct RuleInitWrapper<T> {
    rule: T,
}

/// Hot potato wrapper used during *rule updates*.
/// Must be resolved (unwrapped) in the same transaction.
public struct RuleUpdateWrapper<T> {
    rule: T,
}

/// Creation flow

/// Create a wrapper for rule initialization.
public(package) fun new_init<T>(rule: T): RuleInitWrapper<T> {
    RuleInitWrapper { rule }
}
```

Remediation

Adding a token type parameter to `RuleInitWrapper` and `RuleUpdateWrapper`.

Patch

Resolved in [99fe56d](#).

Improper SetServiceOwner Access Control

LOW

OS-MSC-ADV-05

Description

`trust_service::set_service_owner` assumes that only the current `Master` holds the `SetServiceOwner` ability, since it blindly removes the `Master` role from `ctx.sender` before assigning it to the new owner. If another role is later granted `SetServiceOwner`, a non-`Master` caller may invoke this function. In that case, `internal_remove_role<T, Master>(self, ctx.sender(), ...)` will fail (as caller is not `Master`), resulting in an abort and breaking the ownership transfer flow.

```
>_ move/securitize/sources/trust_service.move
```

RUST

```
public fun set_service_owner<T>(  
    self: &mut Auth<T>,   
    owner: address,   
    version: &Version,   
    ctx: &mut TxContext,   
) {   
    version.check_is_valid();   
    assert!(ctx.sender() != owner, ESelfTransferNotAllowed);   
    assert!(owner_has_ability<T, SetServiceOwner>(self, ctx.sender()), ENotEnoughPermissions);   
    internal_remove_role<T, Master>(self, ctx.sender(), ctx);   
    internal_assign_role<T, Master>(self, owner, ctx);   
}
```

Remediation

Explicitly restrict `SetServiceOwner` to the `Master` role only, ensuring the caller is always the current `Master` and the logic remains valid.

Patch

Resolved in [ff2dba7](#).

Inconsistency in Flowback Activation Logic LOW

OS-MS-C-ADV-06

Description

There is a semantic inconsistency between `flowback_restriction::is_active` and the enforcement logic in `validate_rule`. In `validate_rule`, a value of `block_flowback_end_time_ms == 0` is treated as an always-active restriction (`restriction_active = (end == 0) || ...`), implying transfers are permanently restricted in that case. However, `is_active` returns `false` when `end == 0`, effectively signaling that the restriction is inactive. This inconsistency may result in incorrect assumptions by integrators or off-chain services.

```
>_ move/securitize/sources/rules/flowback_restriction.move
```

RUST

```
/// Check if flowback restriction is currently active
public fun is_active(rule: &FlowbackRestriction, clock: &Clock): bool {
    if (rule.block_flowback_end_time_ms == 0) {
        return false
    };

    let current_time = clock.timestamp_ms();
    current_time < rule.block_flowback_end_time_ms
}
```

Remediation

Ensure `is_active` returns `true` when `end == 0`.

Patch

Resolved in [5f064bf](#).

Unvalidated Role Cleanup Ability LOW

OS-MSC-ADV-07

Description

`remove_role_type` accepts an arbitrary generic `A` and never verifies that it is the legitimate setter ability associated with role `R`. As a result, an authorized caller may remove an unused role type while supplying a critical unrelated ability such as `SetAbilities`, resulting in that ability to be removed from `Master`. It allows privileged users to corrupt the RBAC configuration during role cleanup.

Remediation

Ensure that `remove_role_type` verifies that the provided `A` matches the role's registered management ability before mutating `Master`'s permissions.

Patch

Resolved in [ff2dba7](#).

Missing Zero Check on TreasuryCap Total Supply

LOW

OS-MSC-ADV-08

Description

`setup` does not require `treasury_cap.total_supply` to be zero currently. This is important because the DS registry, compliance counters, wallet balances, and issuance history all start from zero and assume no tokens exist yet. If setup were allowed on a token with existing supply, PAS/coin balances will already be live while the DS registry will show no investors or balances.

Remediation

Require `treasury_cap.total_supply` to be zero in `setup`.

Patch

Resolved in [237ceb8](#).

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MSC-SUG-00	In the <code>seize</code> flow, the recipient is expected to be a special issuer wallet, so the regular-wallet branch should never be reachable, and may be removed.
OS-MSC-SUG-01	<code>get_attribute_value</code> and <code>get_attribute_expiration</code> claim to return 0 for missing investors, but actually abort due to <code>borrow</code> being called first. This creates a mismatch between documentation and real behavior.
OS-MSC-SUG-02	<code>resolve_issuance_time</code> allows future issuance timestamps when back-dating is enabled, due to missing validation against the current time.
OS-MSC-SUG-03	<code>set_special_wallet</code> allows <code>wallet_type = 0</code> , which may act as an invalid or uninitialized value. This may break wallet classification logic and result in inconsistent handling of special wallets.
OS-MSC-SUG-04	The percentage-based limit is computed utilizing the pre-transaction total, creating rounding errors and off-by-one inconsistencies during validation. This may result in unintended limit bypasses or false rejections near boundary conditions.
OS-MSC-SUG-05	Recommendations for more flexible approval rules, clearly distinguishing missing data from a real default status, and improved access control logic.
OS-MSC-SUG-06	Suggestions to remove redundant code, ensure alignment between solidity and move implementations, and to add functionality to update investor counts when compliance metadata changes.

Unreachable Seize Recipient Branch

OS-MSC-SUG-00

Description

In the intended `seize` flow, the recipient should always be an issuer-designated special wallet, so the `investors.is_wallet(to_address)` branch should be unreachable. As a result, this `if` condition is essentially dead code and may be removed.

```
>_ move/securitize/sources/ds_token.move
```

RUST

```
public fun seize<T>(  
    auth: &Auth<T>,   
    investors: &mut InvestorInfo<T>,   
    policy: &Policy<Balance<T>>,   
    mut request: Request<ClawbackFunds<Balance<T>>>,   
    to: &Account,   
    to_address: address,   
    reason: String,   
    version: &Version,   
    ctx: &mut TxContext,   
) {   
    [...]   
    if (investors.is_wallet(to_address)) {   
        let id = investors.get_investor_id_by_wallet(to_address);   
        let total_balance = investors.investor_wallet_balance_total(id);   
        investors.update_investor_total_balance(  
            id,   
            try_from_u256_to_u64!((total_balance as u256) + (value as u256)),   
        );   
   
        let wallet_balance = investors.investor_wallet_balance(to_address);   
        investors.update_wallet_balance(  
            to_address,   
            try_from_u256_to_u64!((wallet_balance as u256) + (value as u256)),   
        );   
    }   
    [...]   
}
```

Remediation

Remove the unreachable branch or modify the condition.

Patch

Resolved in [8ed3c20](#).

Docstring and Implementation Mismatch

OS-MSC-SUG-01

Description

In `registry_service`, there is a mismatch between the documented behavior and the actual implementation of `get_attribute_value` and `get_attribute_expiration`.

Both functions claim to return zero if the investor or attribute does not exist, but they immediately call `investor_info.investors.borrow(investor_id)`. This will abort if the investor is not present, so the "return 0" path is never reached for missing investors.

```
>_ move/securitize/sources/registry_service.move
```

RUST

```
/// Returns the value of a specific attribute for an investor.
/// Returns 0 if the investor or attribute does not exist.
public fun get_attribute_value<T>(
    investor_info: &InvestorInfo<T>,
    investor_id: String,
    attribute_id: u64,
): u64 {
    let investor = investor_info.investors.borrow(investor_id);
    if (!investor.attributes.contains(attribute_id)) {
        return 0
    };
    investor.attributes.borrow(attribute_id).value
}

/// Returns the expiration timestamp of a specific attribute for an investor.
/// Returns 0 if the investor or attribute does not exist.
public fun get_attribute_expiration<T>(
    investor_info: &InvestorInfo<T>,
    investor_id: String,
    attribute_id: u64,
): u64 {
    let investor = investor_info.investors.borrow(investor_id);
    if (!investor.attributes.contains(attribute_id)) {
        return 0
    };
    investor.attributes.borrow(attribute_id).expiration
}
```

Remediation

Ensure to either update the implementation to match the docstrings or revise the docstrings to accurately reflect the actual behavior.

Patch

Resolved in [b8720fd](#).

Missing Issuance Time Validation

OS-MSC-SUG-02

Description

`compliance_service::resolve_issuance_time` accepts `issuance_time_ms` as-is when back-dating is allowed, without checking whether it exceeds `current_time_ms`. This implies a caller may supply a future timestamp, which becomes the effective issuance time. Such future-dated issuance may break downstream time-based logic.

```
>_ move/securitize/sources/compliance_service.move
```

RUST

```
fun resolve_issuance_time<T>(  
  config: &ComplianceConfig<T>,  
  issuance_time_ms: u64,  
  current_time_ms: u64,  
) : u64 {  
  if (!has_rule<T, BackdatingIssuance>(config)) {  
    return current_time_ms  
  };  
  
  let rule: &BackdatingIssuance = config  
    .rules_bag  
    .borrow(type_name::with_defining_ids<BackdatingIssuance>());  
  
  if (rule.is_backdating_allowed()) issuance_time_ms else current_time_ms  
}
```

Remediation

Enforce that the issuance time is never greater than the current block time.

Patch

This issue was acknowledged.

Possibility to Assign Wallet Type to Zero

OS-MSC-SUG-03

Description

`registry_service::set_special_wallet` does not validate `wallet_type`, allowing a value of 0 to be assigned. Since 0 is often treated as a default or uninitialized value, it may conflict with logic that expects valid wallet types to be non-zero. If downstream checks rely on non-zero wallet types to determine behavior, a zero value will result in incorrect handling of the wallet, resulting in undefined behavior.

```
>_ move/securitize/sources/registry_service.move
```

RUST

```
/// Sets a special wallet type for a wallet address.  
public(package) fun set_special_wallet<T>(   
    investor_info: &mut InvestorInfo<T>,   
    wallet: address,   
    wallet_type: u64,   
) {   
    investor_info.special_wallets.add(wallet, wallet_type);   
    investor_info.special_wallets_balance.add(wallet, 0);   
}
```

Remediation

Adding a validation to reject `wallet_type == 0`.

Patch

This issue was acknowledged.

Incorrect Percentage Limit Calculation

OS-MSC-SUG-04

Description

`investor_limits::effective_us_limit` derives the percentage-based cap utilizing the current (pre-transaction) total investor count. This creates edge-case inconsistencies when evaluating an operation that adds a new investor, since the constraint is computed against an outdated denominator. At low totals, integer division `(floor(total * percentage / 100))` may round down to zero, which may be treated as unlimited. Near boundary conditions, this may also result in false rejections because the computed cap is lower than what would be valid post-transaction.

```
>_ move/securitize/sources/rules/investor_limits.move
```

RUST

```
public fun effective_us_limit(absolute_limit: u64, max_percentage: u64, total: u64): u64 {  
    // If percentage is not configured, return absolute  
    if (max_percentage == 0) {  
        return absolute_limit  
    };  
  
    let percentage_limit = (((total as u128) * (max_percentage as u128)) / 100) as u64;  
  
    // If absolute is not configured, return percentage  
    if (absolute_limit == 0) {  
        return percentage_limit  
    };  
  
    // Both set, return min  
    if (absolute_limit < percentage_limit) {  
        absolute_limit  
    } else {  
        percentage_limit  
    }  
}
```

Remediation

Utilize `total + 1` when calculating the percentage cap to align the limit with the post-operation state, ensuring correct enforcement. Retaining the fallback to the absolute limit when the computed percentage is zero avoids unintended scenarios while preserving expected behavior.

Patch

This issue was acknowledged.

Code Refactoring

OS-MSC-SUG-05

Description

1. In `ds_token::burn`, the call to `request.approve(ClawbackApproval<T>())` assumes that only this single approval is required to satisfy the policy. If the PAS policy is later extended to require multiple approvals, `burn` will still provide just one approval and proceed to `resolve`. Thus, the request will be under-approved and `resolve` may abort. Design the approval system to be more flexible for future additions.

```
>_ move/securitize/sources/ds_token.move
```

RUST

```
public fun burn<T>(
  treasury: &mut Treasury<T>,
  auth: &Auth<T>,
  investors: &mut InvestorInfo<T>,
  policy: &Policy<Balance<T>>,
  mut request: Request<ClawbackFunds<Balance<T>>>,
  reason: String,
  version: &Version,
  ctx: &mut TxContext,
) {
  [...]
  request.approve(ClawbackApproval<T>());
  let balance = pas::clawback_funds::resolve(request, policy);
  [...]
}
```

2. `registry_service::get_attribute_value` returns zero for missing attributes, but 0 also represents the default `PENDING` value. This introduces ambiguity in downstream logic that relies on attribute values for compliance decisions. Return `option::none` instead to explicitly indicate the absence of an attribute.

```
>_ move/securitize/sources/registry_service.move
```

RUST

```
public fun get_attribute_value<T>(
  investor_info: &InvestorInfo<T>,
  investor_id: String,
  attribute_id: u64,
): u64 {
  let investor = investor_info.investors.borrow(investor_id);
  if (!investor.attributes.contains(attribute_id)) {
    return 0
  };
  investor.attributes.borrow(attribute_id).value
}
```

3. In `trust_service::owner_has_ability`, add an early return for a `Master` role to grant unconditional access, bypassing all ability checks. This simplifies role management by removing the requirement to explicitly assign every ability.

Remediation

Incorporate the above refactors.

Patch

1. Issue #1 was acknowledged.
2. Issue #2 was acknowledged.
3. Issue #3 was acknowledged.

Code Maturity

OS-MSC-SUG-06

Description

1. In `compliance_service::unregister_rule`, the `contains(&rule_type)` check already guarantees the rule exists in `self.rules`, so `index_of` will always return `exists = true`. The second assertion (`assert!(exists, ERuleNotFound)`), therefore, re-checks an invariant that is already established, rendering it redundant. Remove the redundant check.

```
>_ move/securitize/sources/compliance_service.move RUST

public fun unregister_rule<T, R: store + drop>(
    self: &mut ComplianceConfig<T>,
    auth: &Auth<T>,
    version: &Version,
    ctx: &TxContext,
) {
    version.check_is_valid();
    assert!(auth.owner_has_ability<T, UnregisterRule>(ctx.sender()), ENotAuthorized);
    let rule_type = type_name::with_defining_ids<R>();
    // Check if rule exists
    assert!(self.rules.contains(&rule_type), ERuleNotFound);
    // Remove the typename from the rules vector
    let (exists, idx) = self.rules.index_of(&rule_type);
    assert!(exists, ERuleNotFound);
    self.rules.remove(idx);
    [...]
}
```

2. In solidity (`isMaximumHoldingsPerInvestorOk`), the condition `balance_after >= max` evaluates to `true` when the limit is reached or exceeded. However, in move (`validate_max_holdings`), the check `balance_after <= max` enforces a strict invariant, allowing balances up to and including the maximum, and aborting only when the limit is exceeded. This creates inconsistent behavior where boundary-equal transfers may be flagged or blocked in Solidity but succeed in the Move implementation. Ensure alignment between the two implementations to ensure consistency.
3. `InvestorInfo` maintains aggregate state, but functions such as `set_attribute` and `set_country_compliance` may change an investor's classification after initial registration. If these updates do not adjust the corresponding aggregates, the system may become internally inconsistent, where stored counts no longer reflect actual investor data. To address this, implement functionality to update investor counts when compliance metadata changes.

Remediation

Implement the above-mentioned suggestions.

Patch

1. Issue #1 resolved in [b8720fd](#).
2. Issue #2 resolved in [679b6a5](#).
3. Issue #3 resolved in [99fe56d](#).

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the [General Findings](#).

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
 - Improperly designed economic incentives leading to loss of funds.
-

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
 - Exploitation involving high capital requirement with respect to payout.
-

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
 - Forced exceptions in the normal user flow.
-

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.
-

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
 - Improved input validation.
-

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.